

SYSTEM DESIGN

≈ PARKING LOT ≈

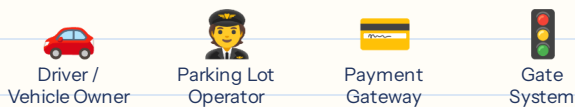
1. Problem Statement

Design a Parking Lot System that manages vehicle entry, exit, parking space allocation, billing and payment. The system should be **efficient, scalable, reliable** and **easy to maintain**.

2. Functional Requirements

- ★ **Vehicle Entry** — System should allow vehicles to enter the parking lot.
- ★ **Spot Allocation** — System should assign an available parking spot.
- ★ **Ticket Generation** — System should generate a ticket with entry time and assigned spot.
- ★ **Vehicle Exit** — System should allow vehicles to exit after payment.
- ★ **Fee Calculation** — System should calculate parking fee based on duration.
- ★ **Payment Processing** — System should accept payment and generate receipt.
- ★ **Spot Release** — After successful payment, the spot should be released.
- ★ **Support Multiple Vehicle Types** — Car, Bike, Truck, SUV, EV, etc.
- ★ **Support Multiple Floors** — Handle multiple floors / levels in the parking lot.

Actors



Goal

Efficiently manage parking spots and provide a smooth experience for users from entry to exit.

Non-Functional Requirements

- ★ **Scalability** — thousands of vehicles, multiple lots.
- ★ **Performance** — quick spot allocation & billing.
- ★ **Security** — secure payment & vehicle data.
- ★ **Availability** — highly available (24/7).
- ★ **Reliability** — no data loss, consistent billing.
- ★ **Maintainability** — easy to extend & modify.

4. Assumptions

- ★ Each vehicle has a unique number plate.
- ★ Payment is required before exit.
- ★ Surveillance / security system is out of scope.
- ★ A vehicle can have only one active ticket at a time.
- ★ Monthly subscriptions are out of scope (can be extended).
- ★ Gates have cameras for number plate recognition (ANPR).

5. Vehicle & Spot Types

Vehicle Types

Car Bike SUV Truck EV (Electric)

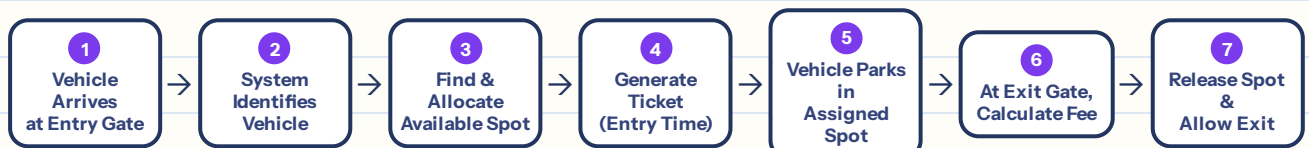
Each type has different size requirements.

Spot Types

Compact (Small) Regular (Medium) Large

EV Charging Reserved (VIP/Disabled/Ladies)

6. High Level Parking Flow

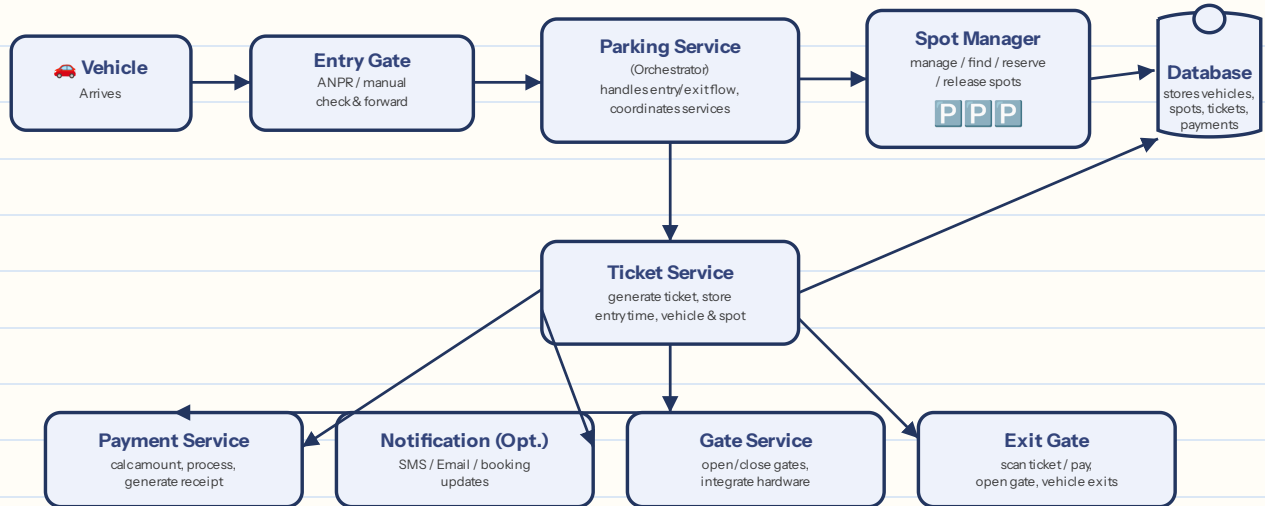


SYSTEM DESIGN

≈ PARKING LOT ≈

2. High Level Architecture (HLD)

At a high level, the Parking Lot System is composed of the following major components.



Component Responsibilities

Component	Responsibility
① Entry Gate	First point of contact. Captures vehicle info, checks availability, forwards request.
② Parking Service	The brain of the system — orchestrates the complete flow, coordinates all other services.
③ Spot Manager	Maintains all parking spots & status, finds the best available spot based on strategy.
④ Ticket Service	Generates a unique ticket with entry time, vehicle & spot info. Used during exit.
⑤ Payment Service	Calculates parking fee based on duration & rules, processes payment, generates receipt.
⑥ Gate Service	Controls physical gates (barriers), integrates with hardware to open/close gates.
⑦ Exit Gate	Handles exit flow — validates ticket & payment, opens gate for vehicle to leave.
⑧ Database	Central storage for all system data. Ensures consistency.

SYSTEM DESIGN

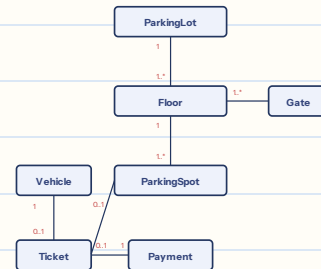
≈ PARKING LOT ≈

3. Core Entities & Relationships

Let's identify the main objects (entities) in our system and how they are related.

- ① **ParkingLot** — represents the entire lot; can have multiple floors.
- ② **Floor** — each floor contains multiple parking spots.
- ③ **ParkingSpot** — individual spot where a vehicle can be parked.
- ④ **Vehicle** — vehicle that enters and parks in the lot.
- ⑤ **Ticket** — issued to vehicle at entry; used for exit & billing.
- ⑥ **Gate** — entry and exit gates for vehicles.
- ⑦ **Payment** — payment details for parking fee.

Entity Relationship Overview



Entity Details

Entity	Attributes	Responsibilities
ParkingLot	id, name, address, totalFloors, totalSpots, operatorInfo	Add/remove floors, get overall availability, provide configuration
Floor	id, floorNumber, spots, totalSpots, availableSpots	Add/remove spots, get available spots, manage floor status
ParkingSpot	id, spotNumber, type, status, vehicle	Mark available/occupied, assign/remove vehicle, validate compatibility
Vehicle	id (numberPlate), type, ownerName, contact	Provide vehicle details, vehicle type validation
Ticket	id, entryTime, vehicle, spot, status	Store entry info, calculate duration, link with payment
Gate	id, gateNumber, type, status, hardwareId	Open/close gate, validate entry/exit
Payment	id, ticket, amount, method, status, receiptNo	Process payment, generate receipt



Why these entities?

These objects capture the core information needed to manage parking operations from entry to exit.



Things to Remember

These are domain objects (not tables) — will map to DB tables later. Keep entities small with a single responsibility.

SYSTEM DESIGN

≈ PARKING LOT ≈

4. Parking Spot Allocation

One of the core challenges is to quickly find the best available spot for a vehicle based on its type and the chosen strategy.

4.1 How Do We Find A Spot?

- ① Identify vehicle type (Car/Bike/SUV/EV etc.)
- ② Get list of available spots that are compatible.
- ③ Apply allocation strategy to pick the best spot.
- ④ Reserve the selected spot (mark as "occupied").
- ⑤ Return the spot details to the system.

Vehicle	Compact	Regular	Large	EV Chg.	Bike Spot
Car	✓	✓	✓	-	-
Bike	✓	✓	✓	-	✓
SUV	-	✓	✓	-	-
EV	-	✓	✓	✓	-
Truck	-	-	✓	-	-

↑ Spot Compatibility Matrix

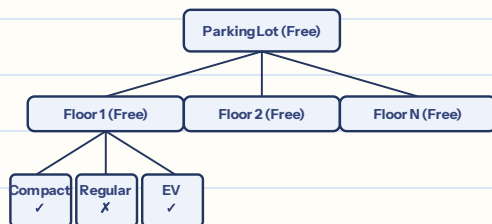
4.2 Allocation Strategies

- ★ **Nearest Spot** — allocate closest spot to entry gate.
- ★ **Best Fit** — choose smallest spot that fits vehicle.
- ★ **VIP / Reserved** — reserve spots for VIP/Ladies/Disabled.

- ★ **First Fit** — scan fixed order, pick first free spot.
- ★ **EV Priority** — prefer EV spots for EVs; fallback if full.

4.3 How Do We Track Available Spots?

Availability is maintained at multiple levels for fast lookup — avoids scanning the entire lot on every arrival.



4.4 Allocation Algorithm (Nearest — Example)

- Input: vehicleType, entryGateId
- Get floors sorted by distance from gate.
- For each floor: get compatible spots — if not empty, return best spot.
- If no spot found on any floor → Parking Full.

Time Complexity: $O(F \times S_f)$ — F = floors, S_f = spots/floor

4.5 Concurrency Challenge

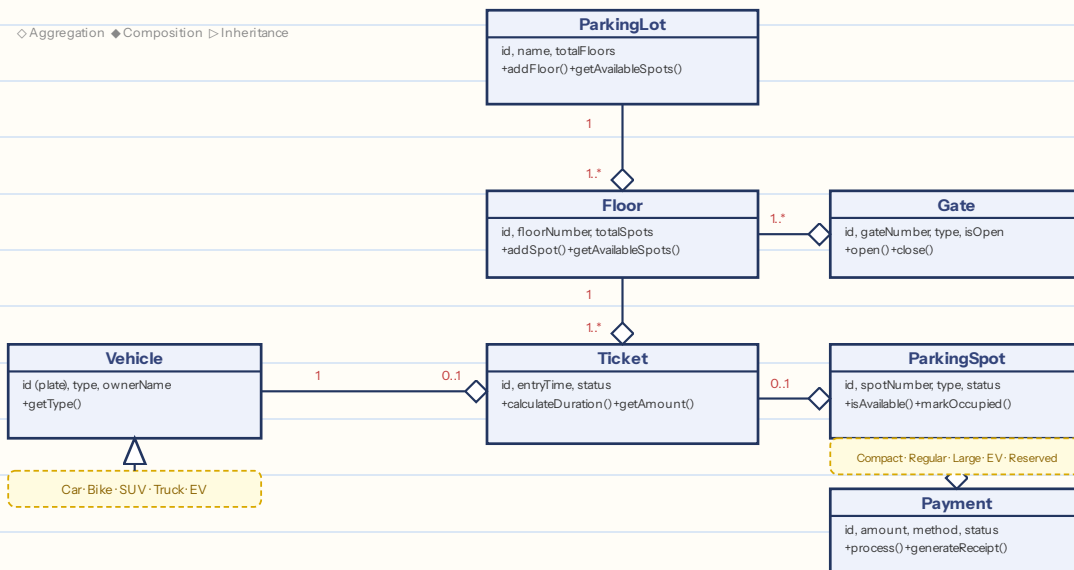
What if two vehicles try to book the same spot at the same time? Both might see it as "free."
Fix: use locking/transactions — the spot is atomically reserved before ticket generation. If anything fails, roll back and release it.

SYSTEM DESIGN

≈ PARKING LOT ≈

5. UML Class Diagram

The main classes, key attributes/methods, and how they relate. **ParkingLot** is the root aggregate — it owns all Floors.



Enums

VehicleType {CAR, BIKE, SUV, TRUCK, EV} SpotType {COMPACT, REGULAR, LARGE, EV, RESERVED}
SpotStatus {FREE, OCCUPIED, RESERVED, DISABLED}
TicketStatus {ACTIVE, CLOSED, CANCELLED}
PaymentMethod {CASH, CARD, UPI, WALLET}
PaymentStatus {SUCCESS, FAILED, PENDING, REFUNDED}
GateType {ENTRY, EXIT}

✓ Key Takeaways

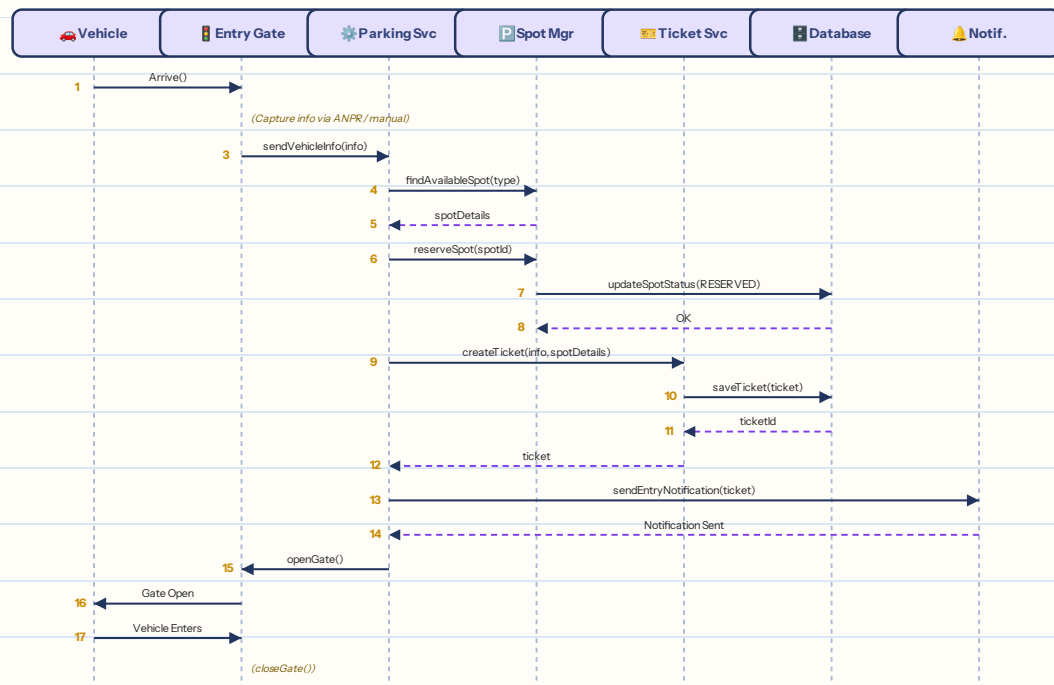
Classes are grouped by responsibility. ParkingLot ◇ Floor (1 to Many). Floor ◇ ParkingSpot (1 to Many).

SYSTEM DESIGN

≈ PARKING LOT ≈

6. Entry Flow — Sequence Diagram

This diagram shows the end-to-end flow when a vehicle enters the parking lot.



Step-by-Step

① Vehicle arrives at entry gate. ② System identifies vehicle (ANPR/manual). ③ Parking Service requests Spot Manager to find best spot. ④ Spot reserved & status updated in DB. ⑤ Ticket generated & saved. ⑥ Ticket issued, entry notification sent (optional), gate opens. ⑦ Vehicle enters & gate closes.

Failure Scenarios

Parking Full — no available spot · **Gate Failure** — gate does not open · **Ticket Generation Failure** — DB/service down

Quick Summary

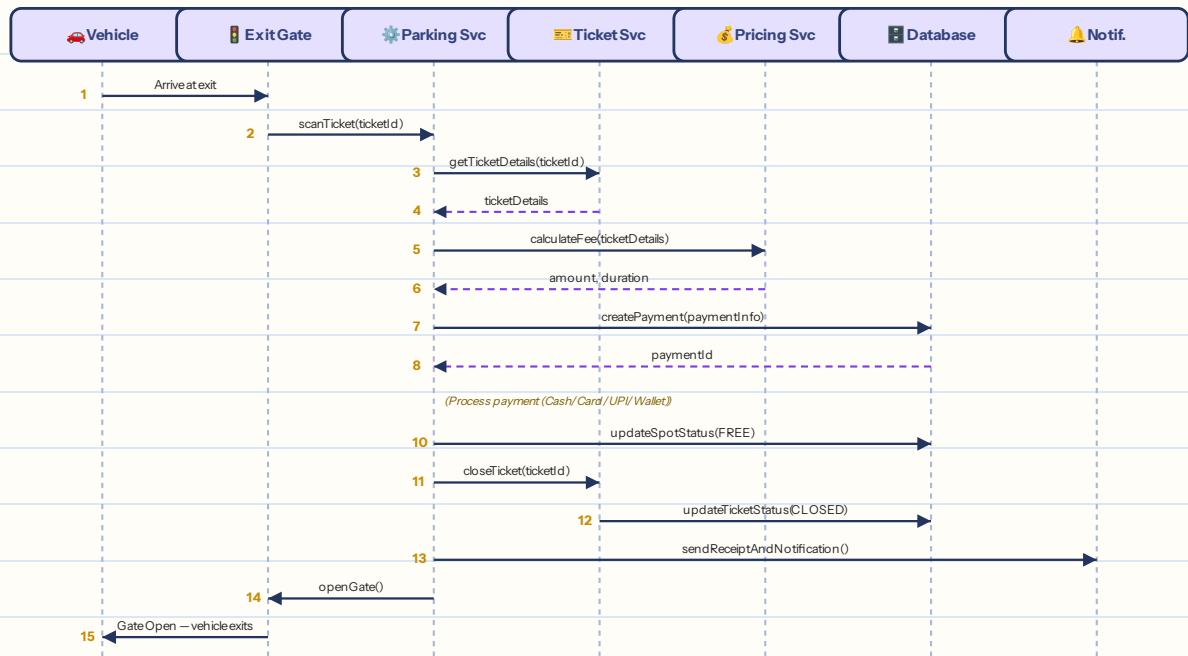
Identify Vehicle → Find Spot → Reserve → Issue Ticket → Open Gate

SYSTEM DESIGN

≈ PARKING LOT ≈

7. Exit & Payment Flow — Sequence Diagram

This diagram shows the flow when a vehicle wants to exit the parking lot.



Key Points

Ticket acts as the bridge between entry & exit. Fee is calculated from duration, vehicle type & pricing rules. Only after successful payment is the spot released and ticket closed. All transactions are recorded for audit/reports.

Payment Methods

Cash · Card · UPI · Wallet

Failure Scenarios

Payment Failed → show error · Lost Ticket → verify by plate · Gate/Hardware Failure → manual mode · Network Down → offline mode

SYSTEM DESIGN

≈ PARKING LOT ≈

8. Design Patterns Used

Design patterns help us build a flexible, maintainable and extensible Parking Lot System.

① Strategy Pattern (Behavioral)

Why? Multiple ways to choose a parking spot or calculate pricing — lets us switch the algorithm at runtime.

How used? Spot allocation strategies (Nearest, FirstFit, BestFit, VIP, EV Priority); pricing strategies (Hourly, Flat, Dynamic, Peak).

SpotSelectionStrategy «interface» → NearestSpotStrategy / FirstFitStrategy / BestFitStrategy

② Factory Pattern (Creational)

Why? We create different objects (Vehicle, ParkingSpot, Ticket, Payment) based on input — factory hides creation logic.

How used? Create Vehicle by type, ParkingSpot by type/size, Payment by selected method.

VehicleFactory.createVehicle(type) → Car / Bike / Truck ...

③ Singleton Pattern (Creational)

Why? We need only one instance of ParkingLotManager holding configuration, counters & global utilities.

How used? Single instance of ParkingLotManager / ConfigurationManager; thread-safe access to shared resources.

ParkingLotManager.getInstance() → one instance app-wide

④ State Pattern (Behavioral)

Why? A Spot / Ticket can be in different states, and behavior changes based on state — avoids large if-else chains.

How used? Spot states: FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE. Ticket states: ACTIVE, CLOSED, CANCELLED.

SpotState «interface» → FreeState / ReservedState / OccupiedState



Why Patterns Matter

Encourages loose coupling and high cohesion.



In Parking Lot Design

Strategy → Algorithms (allocation, pricing) · Factory → Object creation · Singleton → Shared resources · State → Spot/Ticket lifecycle

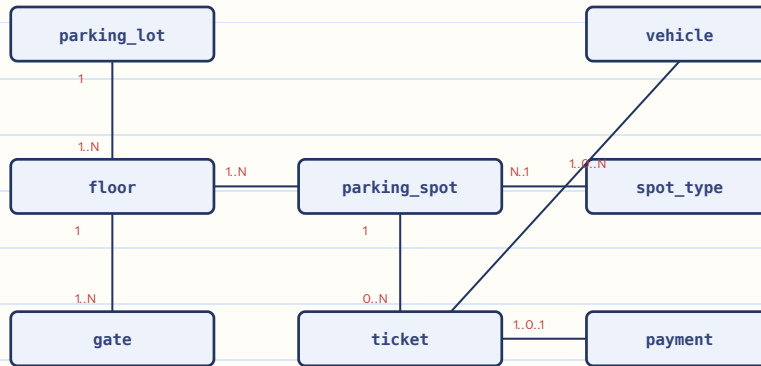
SYSTEM DESIGN

≈ PARKING LOT ≈

9. Database Design

We will use a relational database (MySQL) to store all core data.

★ ER Diagram (High Level)



★ Tables & Key Columns

Table	Key Columns	Notes
parking_lot	lot_id (PK), name, address, total_floors	Auto increment PK
floor	floor_id (PK), lot_id (FK), floor_number, total_spots	FK → parking_lot.lot_id
parking_spot	spot_id (PK), floor_id (FK), spot_number, type_id (FK), status	status ENUM: FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE
spot_type	type_id (PK), name, min_width, description	name UNIQUE
gate	gate_id (PK), lot_id (FK), gate_number, gate_type, status	gate_type ENUM: ENTRY, EXIT
vehicle	vehicle_id (PK), number_plate (UNIQUE), vehicle_type, owner_name	vehicle_type ENUM: CAR, BIKE, SUV, TRUCK, EV
ticket	ticket_id (PK), vehicle_id (FK), spot_id (FK), entry_gate_id (FK), entry_time, status	status ENUM: ACTIVE, CLOSED, CANCELLED
payment	payment_id (PK), ticket_id (FK, UNIQUE), amount, method, status	status ENUM: SUCCESS, FAILED, REFUNDED

★ Important Indexes

idx_vehicle_plate (vehicle.number_plate) · idx_spot_status (parking_spot.status) · idx_ticket_entry_time (ticket.entry_time) · idx_payment_ticket (payment.ticket_id)

Design Notes

These are domain objects mapped to relational tables. Enum columns keep status values consistent and query-friendly across the system.

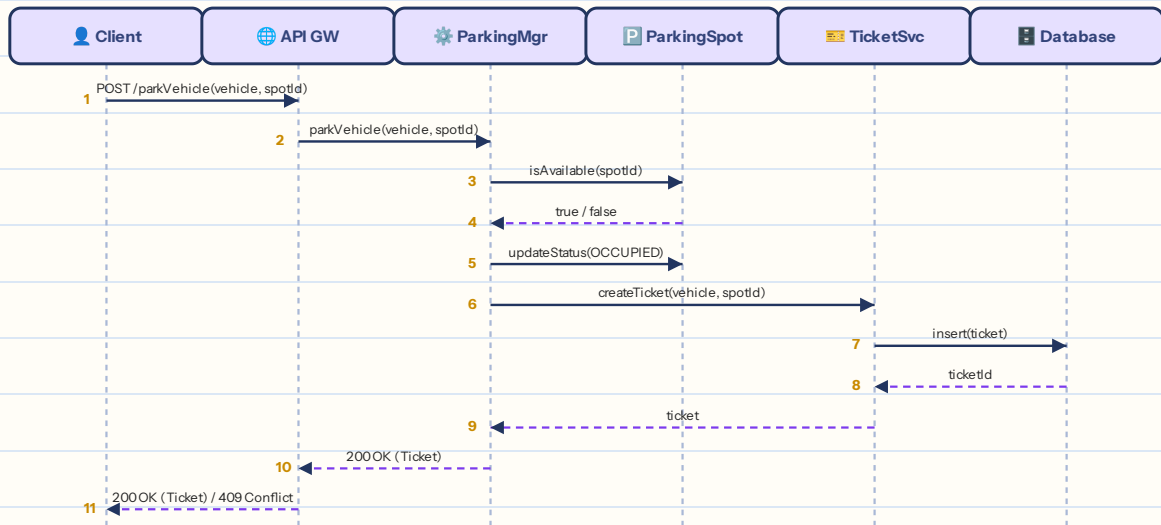
SYSTEM DESIGN

≈ PARKING LOT ≈

10. APIs & Core Operations

API	Purpose	Input	Output
getAvailableSpots(spotType)	Get available spots for a type	spotType	List<ParkingSpot>
parkVehicle(vehicle, spotId)	Park a vehicle in a spot	vehicle, spotId	Ticket
unparkVehicle(ticketId)	Remove vehicle and close ticket	ticketId	Payment, Ticket
getParkingFee(ticketId)	Calculate fee for a ticket	ticketId	amount
getTicketDetails(ticketId)	Get details of a ticket	ticketId	Ticket
getRevenue(from, to)	Get revenue between dates	fromDate, toDate	amount

★ Sample API Flow — parkVehicle(vehicle, spotId)



🚀 Scalability Considerations

- ① Horizontal Scaling — services behind load balancer
- ② DB Scaling — read replicas / sharding
- ③ Caching — spots, pricing, configs
- ④ Async — queues (RabbitMQ/Kafka) for notifications
- ⑤ Monitoring — logs, metrics, alerts

⚠️ Edge Cases to Handle

No spots of any type · Payment failure after parking · Gate hardware failure · Duplicate entry/exit · Lost ticket · Concurrent park/unpark requests · Negative/invalid payment

SYSTEM DESIGN

≈ PARKING LOT ≈

11. Additional Design Details

① Pricing Engine — Calculate fee based on rules

Type	Base Rate	Rules	Example
Hourly	₹20 / hour	Round up to next hour	1h 15m → ₹40
Daily Flat	₹200 / day	Up to 24 hours	Any time → ₹200
Dynamic	₹10 - ₹50 / hour	High demand → high price	Peak hours → ₹50
Subscription	₹1500 / month	Fixed slots / unlimited	Unlimited parking
EV Discount	10% off	—	Total - 10%

② State Transitions (Ticket)



Overstay: If payment fails → remain in PAYMENT_PENDING (extra charges) → send reminder → mark OVERDUE. **Cancel/Error** path → CANCELLED (refund if any).

④ Notifications

Ticket issued · Reminder before exit ·
Payment success/failure · Overstay alerts ·
Offers · System alerts

⑤ Error Handling

No spot → suggest alt lot · Payment fail →
retry/alt method · Hardware fail → manual
mode · Duplicate → idempotency keys

⑥ Security

Auth + rate limiting · Encrypt PII/payments
· RBAC · Audit logs · Prevent
SQLi/XSS/CSRF · PCI-DSS compliance

⑦ High Level Component Diagram



SYSTEM DESIGN

≈ PARKING LOT ≈

12. Non-Functional Requirements

- ✓ **Scalability** — millions of spots & transactions.
- ✓ **Availability** — 99.9% uptime.
- ✓ **Performance** — fast search, booking & payments.
- ✓ **Reliability** — no data loss, safe payments.
- ✓ **Security** — protect user data & payments.
- ✓ **Maintainability** — easy to update & extend.
- ✓ **Auditability** — logs for all critical actions.

 **Key Goals**
Reduce search time · Maximize space utilization · Minimize operational cost · Improve user experience · Ensure data consistency

13. Technology Stack (Example)

Layer	Technology
Frontend	React/Next.js (Web), Flutter (Mobile)
Backend	Node.js / Java / Spring Boot
Database	MySQL (Core), Redis (Cache)
Message Queue	RabbitMQ / Kafka
Cloud & Infra	AWS/GCP (EC2, RDS, S3, ELB)
Monitoring	Prometheus, Grafana, ELK

 **Capacity Estimation (Example)**
Total Lots: 100 cities · Avg spots/lot: 500 · **Total Spots: ~50,000**
Daily Bookings: ~200,000 · Avg Booking: 2 hrs · **Peak QPS: ~500 req/sec**
Data Storage: ~100 GB (initial) · Growth: 20–30% / year

 **Design Challenges**
Real-time availability · Dynamic pricing under demand · Peak traffic & concurrent bookings · Payment failures/refunds · Preventing double booking · Offline gate mode · Fraud prevention

18. Future Enhancements


ML Prediction
demand forecast


Smart
Recommendations


Dynamic Pricing
real-time


ANPR
Integration


Analytics
Dashboard


IoT Sensors
spot detection


Multi-city
expansion


EV Charging
network

Notes compiled from a 12-part System Design series on Parking Lot Systems — @abhi_techhub